

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY

[SMART FUNERAL CLAIMS PLATFORM]

PHASE 3: ANALYSIS PHASE

By

Cainos Emah Mtsweni

402307830

A dissertation submitted in partial fulfilment of the Bachelor of Science in Information Technology

Supervisor: Mr. SilaT

2026

Table of Contents

3.1 INTRODUCTION 3
3.2 Information Gathering Methodology (Observation, participatory, interviews) 3
Observation 3
Participatory 3
Interviews 4
Findings 4
3.3 Analysis of the existing system 4
How the traditional (manual) process works 4
Architecture of simple online claim portals 5
How the Smart Funeral Claims Platform would operate 5
3.3.1 Login 7
3.3.2 Sign up 7
3.3.3 Claim submission 8
3.3.4 Admin Claim Review and Status Update 9
3.4 Data Analysis (Data integrity and constraints) 10
Data structure 10
Relationships between the data 11
Quality issues 12
Rules: 12
Data integrity and constraints 13
Constraints 13
3.5 Weakness of the Current system 14
Weaknesses of the traditional, manual funeral claims process 14
Weaknesses of the Smart Funeral Claims Platform 15
3.6 Analysis of the Proposed System (Functional Requirements) 16
3.7 Non-Functional Requirements 17
3.8 Data modeling for proposed system 18
3.8.1 Entity-Relationship Diagram (ERD) 19
3.8.2 Data Dictionary 20
Users Table 20
Claims Table 21
Documents Table 22
Admins Table 23

Claim_Tracking Table 24
Notifications Table 25
3.8.3 Database Schema 25
Step 1: Beginning from the unnormalized state 25
Step 2: First Normal Form (1NF) 25
Step 3: Second Normal Form (2NF) 26
Step 4: Third Normal Form (3NF) 26
Table relationships summary 26
3.8.4 Data Flow Diagram 27
Conclusion 29
REFERENCES 30

3.1 INTRODUCTION

Phase 3 of the project is the Analysis Phase. The goal of this phase is to gain a better understanding of the problem identified in Phase 1 and planned in Phase 2 without doing any coding yet. In this phase there will be an examination of how the funeral claims are managed using both the manual processes as well as basic online solutions to define requirements of the system that will be created (IBM, 2025).

This phase will cover the approaches that have been used to collect information about the processes, the analysis of the process of claiming that is done manually and online, analysis of the data that will be stored in the system, problems and limitations of existing solutions, functional and non-functional requirements of the proposed solution, as well as data models needed for database design (IBM, 2025).

The main goal of this phase is to create a foundation for further design and development of the platform based on the real-life requirements. As funeral claims are being processed by individuals during one of the most emotionally stressful times in their lives, the proposed system should be user-friendly, secure, reliable, and transparent (IBM, 2025).

3.2 Information Gathering Methodology (Observation, participatory, interviews)

Observation

Traditional process of making funeral claims was studied by analyzing the ways in which the users interact with the funeral insurance company. Usually, a claimant will have to make a trip to a branch or an office in order to receive a checklist of documentation required (which includes such documentation as a certified death certificate, identity documents and policy itself), to fill out physical claim form and submit all documents to a consultant (Asana, 2026). The process of claiming insurance was found to depend very much on paper documents and personal interaction, which may be quite challenging for the family in case they live remotely from the branch and cannot easily visit the office (Asana, 2026).

It was also noticed that after a claim is submitted manually, there is no way for the claimant to monitor the status of their claim aside from contacting the insurer over the phone or visiting the office again (Asana, 2026).

Participatory

In order to investigate the level of participatory practices within online processes in South Africa, websites and applications of some of the companies providing insurance services and

funeral cover were examined. Although all of these websites enable a policyholder to login to his/her account and access the details related to a particular policy, very few companies offer a specific structure that would enable a claimant to file a claim using this structure and follow the status of his/her claim online. In cases when such an opportunity is provided by the company, it usually means filling out a generic contact form on the website or downloading the PDF version of the form to fill it in manually and submit it either via email or in person (Asana, 2026).

It became clear that there is a discrepancy between the requirements of a claimant and what he/she is being provided (Asana, 2026).

Interviews

Informal conversations were conducted with several individuals who had experienced the process of filling a funeral claim, whether in person or on behalf of a relative, as well as individuals who deal with documentation in an administrative position. The goal of such conversations was not to conduct any extensive survey but to learn about the difficulties that people experience with the process from firsthand experience (Asana, 2026).

The main difficulties mentioned were: uncertainty about the duration of the procedure, doubts as to whether all the needed documents had arrived, resubmitting some documents due to losing them, and difficulty in receiving any information without calling repeatedly (Asana, 2026).

Findings

It is evident from the observation, participatory research, and interviews that some of the problems that arise in the process of handling the funeral claims currently include dependency on the use of paper documents, absence of an online central registry, invisibility of the claims processing procedure, and the need for manual work on the part of the administrator. The Smart Funeral Claims Platform solves the above-mentioned issues by providing claimants with one online location for registration, submission of their claims and documents, as well as monitoring the claim process (Asana, 2026).

3.3 Analysis of the existing system

How the traditional (manual) process works

In the manual process, the first step that the claimant takes is that he/she needs to identify what documents he or she should prepare; the documents include certified copies of the death certificate, certified copies of the claimant's identity document, proof of the relationship with

the deceased, and the policy number. Then the claimant submits the documents physically to the branch or electronically by e-mail or courier (IBM, 2025). The administrator then receives the documents, goes through them manually using a check list, and enters the details of the claim into a register or spreadsheet or an internal system that has not been specifically developed for funeral claims (IBM, 2025).

The architecture underlying this manual system is very simple; there is no shared digital record of the claim that the claimant and the administrator both have access to. The tracking that does exist is usually within the insurer and not available to the claimant. As there is no validation at the point of capture, there are often errors such as missing data or handwriting that is hard to read, and these are only found out when an administrator looks through the documentation (IBM, 2025).

Architecture of simple online claim portals

A number of insurers offer a simple online portal where the policyholder logs into and can see his/her insurance policy details. If there is any claims functionality at all, it is likely to be a simple contact or query form rather than any sort of claim submission process. Such portals will not categorize claims by type, will not enable uploading of several documents with proper labeling of document type, and will not inform the claimant about his/her claim status which could be pending, under review or approved (IBM, 2025).

How the Smart Funeral Claims Platform would operate

The Smart Funeral Claims Platform transforms this process by taking it all digital and tailoring it especially for funeral claims. The user registers for an account and logs in with Firebase Authentication (IBM, 2025). After logging in, the user submits a claim by inputting the necessary claim information (like policy number, type of claim, name and relation of the deceased) and uploading documents such as the death certificate and identification documents via the browser interface. The documents are stored safely in Firebase Storage while the claim information, including links to the uploaded documents, is stored in Firestore (IBM, 2025).

Once the claim has been filed, its status becomes “Pending.” The claimant can access his account anytime and check out the current status of his claim. On the other hand, an administrator logs into his admin dashboard, which shows all the claims submitted to him. The administrator then accesses the claim, reviews its documents and updates its status to either “Under Review,” “Approved” or “Rejected.” He may add a comment while changing

the status of the claim. This feature ensures that the claimant does not have to call up the administrator for status updates (IBM, 2025).

The primary processes involved in the operation of the Smart Funeral Claims Platform include the following. Firebase Authentication enables the processes of user sign-up, login, password reset, and access management. Firestore Database saves data in a structured format like users' profiles, claims, and claim tracking history. Firebase Storage saves the files that have been uploaded by the users (IBM, 2025). File uploads in the backend environment are enabled by Multer. Email notifications are sent by either Nodemailer or EmailJS in cases like when there is a change in the status of a claim or when a user sends an enquiry from the contact form.dotenv enables the handling of environment variables in such a way that secret configuration information like API keys is not contained in the source code (IBM, 2025).

The sequence diagrams below illustrate four of the most important processes in the Smart Funeral Claims Platform: logging in, registering a new account, submitting a claim, and the administrator reviewing and updating a claim's status. The key used for these diagrams is shown in Table 1.

Symbol / Element	Name	Description
Colored rectangle at the top	Object symbol	Represents an object, interface or class and shows how it behaves in the system. Attributes are not listed inside this shape.
Vertical bar below an object	Activation box symbol	Represents the period of time that an object is active while completing a task.
Stick figure	Actor symbol	Represents a user, such as a claimant or an administrator, who is external to the system.
Solid arrow	Synchronous message symbol	Used when the sender must wait for a response before the process can continue.
Dashed arrow	Asynchronous return message symbol	Used for messages that are replies to an earlier call.
Rectangle labelled Alt	Alternative combined fragment	Shows two or more possible outcomes of a process, such as a valid and an invalid path.

Table 1: Sequence Diagram Key

3.3.1 Login

After the credentials have been input by a logged-in user, they are sent to Firebase Authentication to check whether they are correct. In the case that the credentials are correct, the user's profile will be retrieved from Firestore, and he/she will be directed to the user's dashboard. In the event that the credentials are wrong, an error message will appear (Lucidchart, 2026).

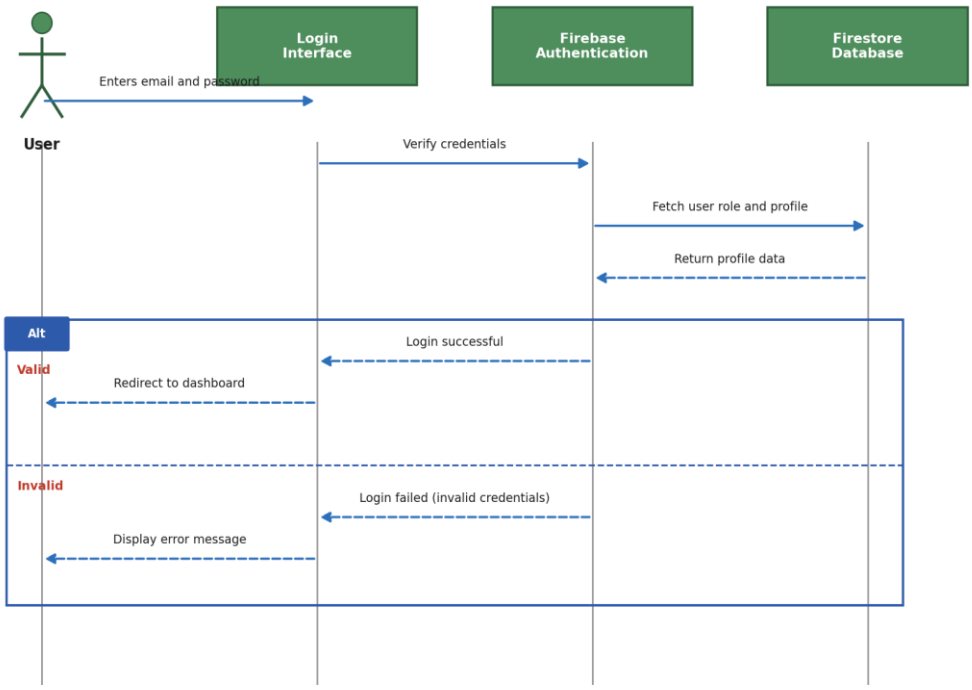


Figure 3.3.1: Login Sequence Diagram

3.3.2 Sign up

The user enters their name, email address, phone number, and password while signing up for an account. This information will then be passed to Firebase authentication where the account will be created. The user's profile will be saved in firestore. In case the credentials entered are correct, then the user will receive a success message along with an email. Otherwise, an error message will be sent to the user (Lucidchart, 2026).

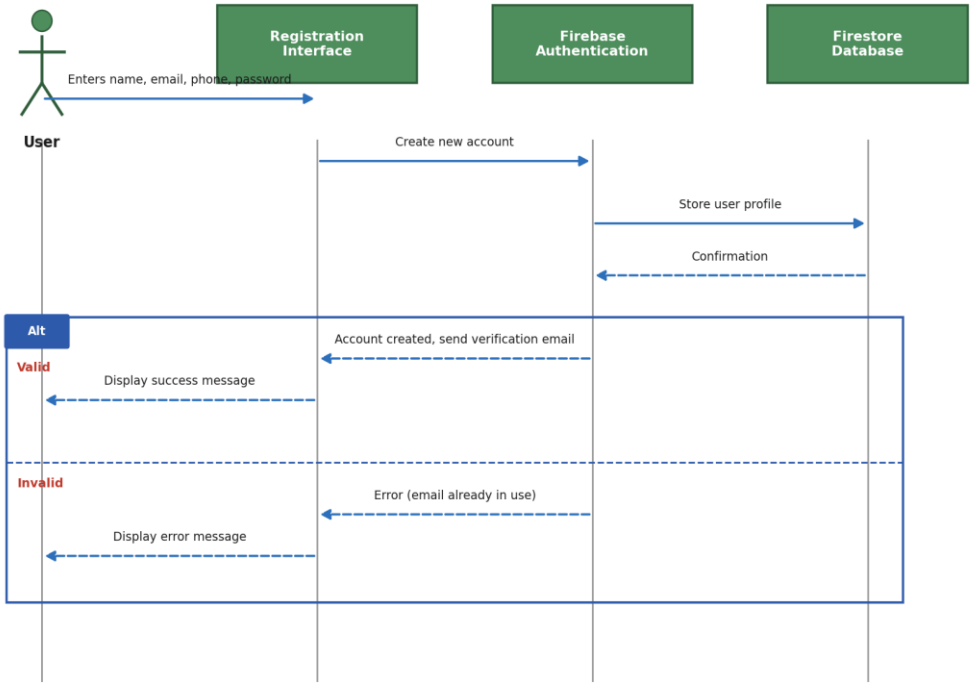


Figure 3.3.2: Registration Sequence Diagram

3.3.3 Claim submission

After the user has submitted their claim, they will fill out the claim form and upload the necessary documents. This claim submission page will upload the files to the Firebase Storage and will receive a file URL as a response. Afterward, it will save the claim record along with the file URL in Firestore. In case everything was fine, then the claim will be saved with Pending status and a successful message will be displayed to the user. In case some fields were left blank, the user will receive an error (Lucidchart, 2026).

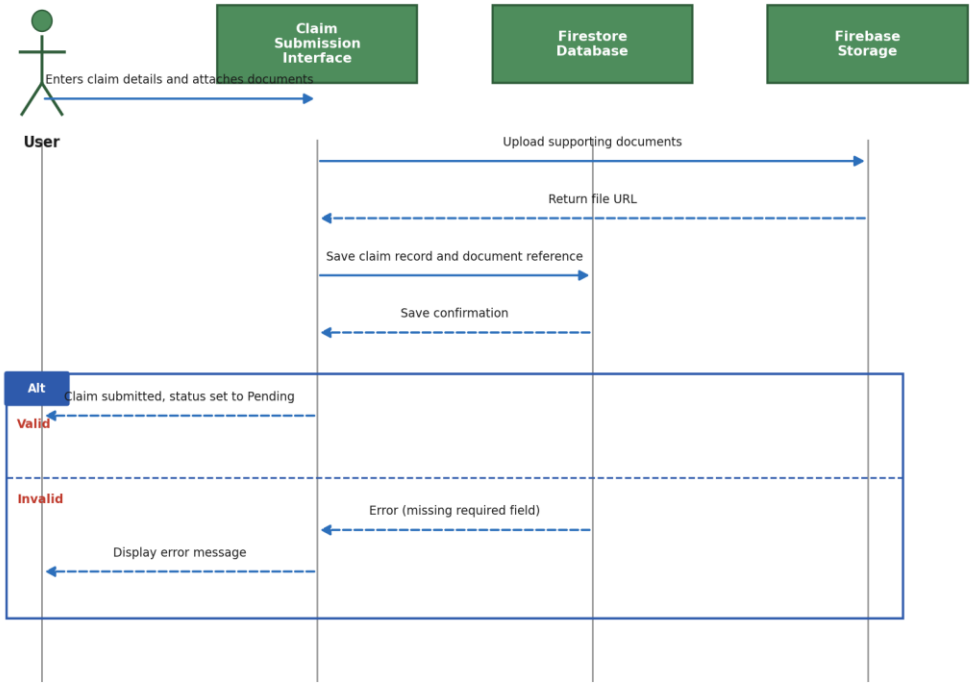


Figure 3.3.3: Claim Submission Sequence Diagram

3.3.4 Admin Claim Review and Status Update

On accessing the claim through the admin panel interface, the claim and associated documents are retrieved from Firestore for the purpose of assessment. Once the documents have been assessed, the status of the claim is updated. This data, along with a log of the action and a comment where applicable, are stored in Firestore. In addition, a notification is generated in order to inform the claimant about the update. In case of any failure during the update process, an error message is displayed to the administrator (Lucidchart, 2026).

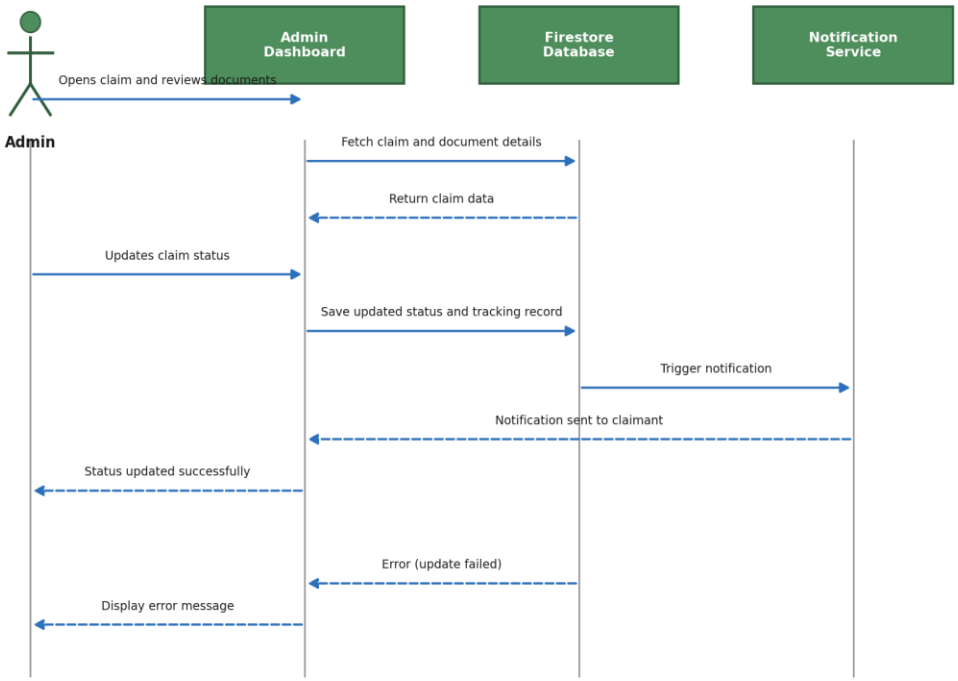


Figure 3.3.4: Admin Claim Review and Status Update Sequence Diagram

3.4 Data Analysis (Data integrity and constraints)

Data structure

The Users table contains the UserId, Name, Surname, Email, Phone, the hash of the password, and the CreatedAt. The UserId is the Primary Key (PK). Email is required for login purposes and notifications on claims. Hence, the email should be unique (Oracle, 2025).

The Claims table contains the following columns: ClaimId, UserId, PolicyNumber, ClaimType, DeceasedName, Relationship, ClaimDate, and Status. ClaimId is the Primary Key (PK), and the UserId is the Foreign Key (FK) as it is a reference to the user that made the claim. The Status column can have only the following values: Pending, Under Review, Approved, or Rejected (Oracle, 2025).

The Documents table contains the following columns: DocumentId, ClaimId, FileName, FileUrl, DocumentType, and UploadedAt. DocumentId is the Primary Key (PK), and the ClaimId is the Foreign Key (FK) as it is a connection between a certain document and the claim (Oracle, 2025).

Administrators are stored with attributes AdminId, Name, Email, Role and CreatedAt. AdminId is the Primary Key (PK). Role attribute is used to define what operations an administrator can perform in the system, thus facilitating scalability of the platform, should different levels of administrative rights become necessary in the future (Oracle, 2025).

Records of Claim_Tracking are stored with attributes TrackingId, ClaimId, AdminId, Status, Comment and UpdatedAt. TrackingId is the Primary Key (PK), whereas ClaimId and AdminId are Foreign Keys (FK). Claim_Tracking table stores the history of all the changes in status of the claim, hence providing the possibility of auditing the progress of the claim (Oracle, 2025).

Notifications are stored with attributes NotificationId, UserId, Message, Date and ReadStatus. NotificationId is the Primary Key (PK) and UserId is a Foreign Key (FK). Notifications table provides the capability of storing information about each message sent to the claimant, for instance in case his/her claim was changed (Oracle, 2025).

Relationships between the data

1. Users: Claims (1:M)

One user is capable of making several claims at different periods, but one claim can only be made by one particular user (Oracle, 2025).

2. Claims: Documents (1:M)

Several documents can be made in connection with one claim, but one document can be connected with only one particular claim (Oracle, 2025).

3. Claims: Claim_Tracking (1:M)

Several entries regarding the claim tracking can be made about one particular claim, but one entry can be related only to one particular claim (Oracle, 2025).

4. Admins: Claim_Tracking (1:M)

One administrator can make several status update entries for different claims (Oracle, 2025).

5. Users: Notifications (1:M)

One user receives several notifications, for example, one for each status update of all his claims (Oracle, 2025).

Quality issues

The following are some fields that must be entered by users when creating a claim: policy number, claim type, name of the deceased, relationship with the deceased, and at least one file as evidence. In case any of these fields have not been filled, the submission form cannot be submitted and a prompt informing the user of the details that have not yet been filled will be displayed. In such a way, the administrator of the application will not have to go through the hassle of dealing with incomplete claims, which was the main issue with the manual system (Asana, 2026).

When registering, the user may enter an email address that does not belong to him or one that contains a typo error. To minimize the risks related to this issue, regular expression (regex) validation and a confirmation email are performed on the email address of the user before he starts using the system (Asana, 2026).

Another way in which users may cause problems is to try uploading a document in a format that the system does not support, or a file that is far too large. This can be dealt with by ensuring that only certain file formats are supported (such as PDF, JPG, and PNG), and also setting a limit on the file size (Asana, 2026).

In addition, there is a possibility that the claimant may input a policy number that does not correspond to an actual policy, either by accident or, more rarely, deliberately. Since the system will not be able to verify that the policy number corresponds to one on file at the insurance company, this would be marked as a manual task for the administrator (Asana, 2026).

Rules:

1. All UserId, ClaimId, DocumentId, AdminId, TrackingId and NotificationId have to be unique since they are all primary keys (Asana, 2026).
2. Each claim has to belong to only one valid user; thus, UserId is a mandatory foreign key on Claims table (Asana, 2026).
3. Each document has to belong to only one claim; hence, ClaimId is a mandatory foreign key on Documents table (Asana, 2026).

4. The Status field on claims has to be one of the predefined status values and should not accept free text (Asana, 2026).
5. The dates and timestamps have to be in a certain standard format (YYYY-MM-DD for dates and YYYY-MM-DD HH:MM:SS for timestamps) in order for the claims to be sortable and reportable (Asana, 2026).
6. The files referenced in the Documents table have to correspond to an existing file in Firebase Storage (Asana, 2026).

Data integrity and constraints

A valid email address and a password should always be present in each user profile in order for an account to be created because these fields are essential for the login process. Regular expressions validation will be used to minimize the chances of storing incorrect data on the registration and claims submission forms. In case the field is really optional, like an extra comment on a claim, it will be indicated as such so that it won't hinder the submission of a claim (Oracle, 2026).

All login details will be managed via Firebase Authentication, where passwords will be stored in hashed form and not in plain text, therefore, the password itself will be accessible to no one, not even the developer. The configuration of Firebase security rules will allow only the claimant themselves to see and modify their claims, and only an administrator account to see all claims and modify the claim statuses (Oracle, 2026).

Since the information stored includes sensitive personal data and medical-related information such as death certificate and identification information for the deceased, Firebase storage and Firestore will be limited by security rules in order to protect the information from any access by an unauthorized person. All interactions between the browser and Firebase will be done over HTTPS (Oracle, 2026).

Constraints

Users' passwords should adhere to Firebase's default password policy. For instance, they should have a minimum length (IBM, 2025).

The platform requires an active internet connection since Firebase is a cloud-based platform. In case of a user having no internet connection, he or she will not be able to log into the system, submit a claim and check the status of his or her claim until the internet connection is

restored. It is an acceptable constraint for a claims platform since claim submission is not an immediate or real-time thing (IBM, 2025).

As the Phase 2 project schedule shows, there are only 17 weeks to plan, design, develop, test, and document this project, which makes it a time-related constraint. The solution here is to implement more complex and/or optional features after the basic registration, login, claim submission, documents upload, tracking, and admin dashboard features start to work properly (IBM, 2025).

One developer works on this project, meaning that this person has to handle the frontend, backend development, Firebase configuration, testing, and documentation tasks. In order to handle this constraint, the developer keeps a simple to-do list where the most important things are done first and also uses already existing and well-documented libraries (such as Multer for files upload) rather than reinventing the wheel. (IBM, 2025).

The platform also uses the Spark plan provided by Firebase. This imposes restrictions on the amount of storage space, number of reads/writes to the database and the amount of bandwidth. This is a reasonable constraint for this student project, but if the platform were to have lots of users, then this issue needs to be reconsidered (IBM, 2025).

3.5 Weakness of the Current system

Weaknesses of the traditional, manual funeral claims process

The current largely paper based process undergone by many of those applying for claims has a number of drawbacks. The reliance on paperwork means that the documents could easily be misplaced, delayed or damaged in transit, slowing down the process even further and adding further stress to a bereaved family (OpenStax, 2025).

This process takes time because of its reliance on physical meetings or letters being posted rather than being instantly submitted online. There is no up to date overview of where the claim is at so there is no easy way for the claimant to know how far through the process he/she is unless they contact the insurer directly. There is also no consolidated database of all claims made (OpenStax, 2025).

Manual data entry is also more likely to involve human errors. For instance, when an officer enters information from a form to a register or spreadsheet, he or she may make a mistake

since there is no automated mechanism for detecting errors or omissions of information. Since everything is done manually, it becomes difficult to establish the identity of those who have reviewed the claims at different times (OpenStax, 2025).

Lastly, it is not easy for the insurance company to collect information about the claims processing period, sources of delays, and satisfaction of claimants due to the non-digital nature of the process (OpenStax, 2025).

Weaknesses of the Smart Funeral Claims Platform

Despite being able to solve most of the issues mentioned above, it would still be wise to highlight some of the limitations of this particular platform so that they can be anticipated (OpenStax, 2025).

Since it is powered by Firebase, this platform requires an active internet connection from the claimant or the administrator. Should either lose their connection, then they will not have access to the platform until such time as the connection gets restored. This is addressed through designing an interface that explains to users when the platform is offline, rather than failing to load (OpenStax, 2025).

Since Firebase Authentication is being used, the security of the user accounts relies on the infrastructure of Firebase itself. Though Firebase is a reputable platform, the drawback is that it means that there is very little control for implementing more advanced encryption systems beyond those provided by Firebase. It is overcome by setting up Firebase security rules appropriately and not using the application for storing extremely confidential information like the details of bank account (OpenStax, 2025).

There is only one person developing the whole application which poses as a disadvantage when it comes to resourcing. It leaves no room for sharing the burden if any unforeseen difficulties arise during development. It can be overcome through making a basic task list, focusing on the key elements first, and using reliable libraries instead of building custom solutions when there are available (OpenStax, 2025).

One potential problem is that some claimants, especially older people or those not as tech-savvy, might have a hard time completing an online form while under stress. The solution is to keep the user interface as straightforward as possible and give helpful hints about how to

fill out the form, as well as to keep a visible way to contact someone if help is needed (OpenStax, 2025).

It is also possible that users will occasionally input wrong data into the system, such as the wrong policy number, even though there is a basic validation system, because the validation system checks only that a field is completed in a proper way, but not whether the information in that field is actually correct (OpenStax, 2025).

Email notifications could fail to reach claimants in some cases; for instance, an email could end up in the spam folder or there could be errors when entering email addresses in the registration process. However, this problem will be prevented by showing the current status of the claim directly within the platform, so that the claimant would not have to rely entirely on an email notification (OpenStax, 2025).

Another limitation of the platform is that it cannot be the final arbitrator whether the claim should be approved and paid. This is due to the fact that there is a need for a human to make decisions in these cases, but it means that the claimant will have to wait for the decision of the administrator (OpenStax, 2025).

Lastly, as the platform is designed to work on multiple devices, there is a possibility that the interface may appear incorrectly on one of the screens. This issue can be mitigated by using responsive design and testing the platform for compatibility with various browsers and devices (OpenStax, 2025).

3.6 Analysis of the Proposed System (Functional Requirements)

1. The system will allow a new user to create an account by entering his/her name, surname, email address, telephone number, and a password.
2. The system will verify the email address of the user before allowing him/her access to the system, through the use of Firebase authentication’s email verification functionality.
3. The system will allow a registered user to sign-in, sign-out, and reset a forgotten password.
4. The system will not allow an unauthorized user to access any claims submission, claims tracking, or administrative pages (GeeksforGeeks, 2026).

5. The system shall enable an authenticated user to file a funeral claim by providing the policy number, claim type, name of the deceased, relationship to the deceased, and claim date (GeeksforGeeks, 2026).
6. The system shall enable a user to upload one or more supporting documents such as death certificate and identification documents for filing a claim.
7. The system shall ensure that all mandatory fields in a claim, along with at least one supporting document, have been provided before a claim can be filed.
8. The system shall automatically assign the status as Pending to any new claim that is filed.
9. The system shall enable a user to view their own claims and their respective status.
10. The system shall enable an administrator to login to the admin panel via a separate interface (GeeksforGeeks, 2026).
11. The system shall enable an administrator to view all filed claims, perform filtering/search and open individual claims for viewing.
12. The system shall permit an administrator to change the status of a claim (e.g. to Under Review, Approved or Rejected), and enter a comment to explain the action.
13. The system shall record the history of all status updates performed on each claim, including the name of the administrator who made the update and the date.
14. The system shall inform a claimant, e.g. via email, about any changes to the status of their claims.
15. The system shall permit an administrator to view and modify the list of registered users where required.
16. The system shall have a contact form enabling a user to submit a generic inquiry, which is then forwarded to the organisation using the email service.
17. The system shall display the privacy policy and ask for the user's consent to the processing of their personal data prior to allowing them to register (GeeksforGeeks, 2026).

3.7 Non-Functional Requirements

1. Performance: The system will load the dashboard page in about 2 to 3 seconds, and will generally react to user actions, like form submission, in a similar timeframe.
2. Security: The system will always store the passwords in hashed form using Firebase authentication, and will never store the password as plain text.

18. Security: The system will use Firebase security rules to ensure that only the claimant can access their own data, and only the administrator account can access all the claims and user information (GeeksforGeeks, 2025).
3. Security: All data sent from the browser to Firebase will be encrypted on the transport level using HTTPS.
4. Usability: The system will provide a simple interface with clearly labelled buttons and elements to ensure that users with little technical background, even elderly users, will be able to submit a claim easily.
5. Reliability: The system will save data appropriately and prevent data loss, for example by verifying that a document has been uploaded before allowing the user to submit a form (GeeksforGeeks, 2025).
6. Availability: The system will be available when all Firebase services are running, and will notify the user if a required service is down.
7. Maintainability: The codes for the system shall be modularly structured such that modifications and improvements will be easily done in the future (for instance, authentication, claims, documents, and admin dashboard).
8. Compatibility: The website shall perform properly when used on common modern web browsers like Chrome, Edge, and Firefox on both desktop and mobile devices.
9. Scalability: The system shall handle an increasing number of users and claims without a significant drop in performance by utilizing the cloud services provided by Firebase.
10. Privacy: The system shall collect only the personal data required for processing the claims and shall not disclose this data to any other party without user's permission.
19. Auditability: The system shall log all changes in the statuses of claims including the responsible administrator and the time of modification (GeeksforGeeks, 2025).

3.8 Data modeling for proposed system

The following is the part which introduces the data models that will serve as the basis for developing the database for the Smart Funeral Claims Platform. The said section contains an ER diagram, a data dictionary which gives a description of all the fields in the tables, the normalization process used in creating the tables, and a data flow diagram (IBM, 2022).

3.8.1 Entity-Relationship Diagram (ERD)

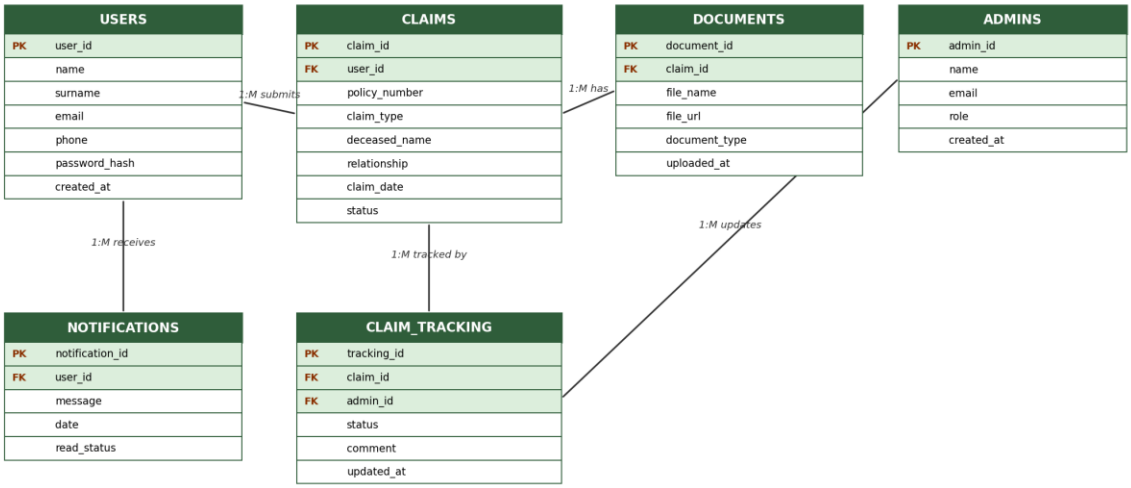


Figure 3.8.1: Entity Relationship Diagram (ERD)

Many claims may be added by a single user, but one claim will be made only by one user (1:M). Many documents can be added by a single claim (1:M), and many claim tracking entries can be added by a single claim (1:M). Also, one administrator can update many tracking entries of different claims (1:M), and many notifications can be sent to one user (1:M) (IBM, 2022).

3.8.2 Data Dictionary

Users Table

Field Name	Data Type	Field Size	Description	Constraints	Example
UserId	UUID	36	Unique identifier automatically generated for each user.	Primary Key. Not Null.	a1b2c3d4-e5f6-7890-a1b2-c3d4e5f67890
Name	String	30	The user's first name.	Not Null.	Emah
Surname	String	30	The user's surname.	Not Null.	Mtsweni
Email	String	50	The user's email address, also used to log in.	Unique. Not Null. Valid email format.	Emah.m@gmail.com
Phone	String	15	The user's contact number.	Not Null.	0821234567
PasswordHash	String	-	The user's password, stored in hashed form by Firebase Authentication.	Not Null. Never stored as plain text.	(hashed value)
CreatedAt	Timestamp	-	The date and time the account was created.	Default = current time.	2026-02-14 09:15:00

Claims Table

Field Name	Data Type	Field Size	Description	Constraints	Example
ClaimId	UUID	36	Unique identifier automatically generated for each claim.	Primary Key. Not Null.	c1a2b3c4-1111-2222-3333-444455556666
UserId	UUID	36	Identifies the user who submitted the claim.	Foreign Key -> Users.UserId. Not Null.	a1b2c3d4-e5f6-7890-a1b2-c3d4e5f67890
PolicyNumber	String	20	The funeral policy number linked to the claim.	Not Null.	POL-2026-00931
ClaimType	String	30	The category of the claim.	Not Null. e.g. Death claim, Cash-back claim.	Death claim
DeceasedName	String	50	The full name of the deceased person.	Not Null.	Sipho Nkosi
Relationship	String	20	The claimant's relationship to the deceased.	Not Null.	Spouse
ClaimDate	Date	-	The date the claim was submitted.	Default = current date.	2026-03-02
Status	String	15	The current status of the claim.	Not Null. One of: Pending, Under Review, Approved, Rejected.	Pending

Documents Table

Field Name	Data Type	Field Size	Description	Constraints	Example
DocumentId	UUID	36	Unique identifier automatically generated for each document.	Primary Key. Not Null.	d1a2b3c4-7777-8888-9999-000011112222
ClaimId	UUID	36	Identifies the claim that the document supports.	Foreign Key -> Claims.ClaimId. Not Null.	c1a2b3c4-1111-2222-3333-444455556666
FileName	String	80	The original file name of the uploaded document.	Not Null.	death_certificate.pdf
FileUrl	String	150	The location of the file in Firebase Storage.	Not Null.	https://firebasestorage.../death_certificate.pdf
DocumentType	String	30	The type of supporting document uploaded.	Not Null. e.g. Death Certificate, ID Copy.	Death Certificate
UploadedAt	Timestamp	-	The date and time the document was uploaded.	Default = current time.	2026-03-02 10:42:11

Admins Table

Field Name	Data Type	Field Size	Description	Constraints	Example
AdminId	UUID	36	Unique identifier automatically generated for each administrator.	Primary Key. Not Null.	ad1a2b3c-5555-6666-7777-888899990000
Name	String	30	The administrator's first name.	Not Null.	Naledi
Email	String	50	The administrator's email address, also used to log in.	Unique. Not Null.	naledi.admin@company.com
Role	String	20	The administrator's access role within the system.	Not Null. e.g. Claims Officer, Super Admin.	Claims Officer
CreatedAt	Timestamp	-	The date and time the admin account was created.	Default = current time.	2026-01-20 08:00:00

Claim_Tracking Table

Field Name	Data Type	Field Size	Description	Constraints	Example
TrackingId	UUID	36	Unique identifier automatically generated for each tracking entry.	Primary Key. Not Null.	tr1a2b3c-1010-2020-3030-404050506060
ClaimId	UUID	36	Identifies the claim that was updated.	Foreign Key -> Claims.ClaimId. Not Null.	c1a2b3c4-1111-2222-3333-444455556666
AdminId	UUID	36	Identifies the administrator who made the update.	Foreign Key -> Admins.AdminId. Not Null.	ad1a2b3c-5555-6666-7777-888899990000
Status	String	15	The claim status recorded at this point in time.	Not Null.	Under Review
Comment	String	150	An optional note explaining the status change.	Optional.	Awaiting certified ID copy.
UpdatedAt	Timestamp	-	The date and time of this status update.	Default = current time.	2026-03-05 14:20:00

Notifications Table

Field Name	Data Type	Field Size	Description	Constraints	Example
NotificationId	UUID	36	Unique identifier automatically generated for each notification.	Primary Key. Not Null.	n1a2b3c4-9999-8888-7777-666655554444
UserId	UUID	36	Identifies the user who the notification was sent to.	Foreign Key -> Users.UserId. Not Null.	a1b2c3d4-e5f6-7890-a1b2-c3d4e5f67890
Message	String	150	The content of the notification.	Not Null.	Your claim status has changed to Approved.
Date	Timestamp	-	The date and time the notification was created.	Default = current time.	2026-03-06 09:00:00
ReadStatus	Boolean	1	Whether the user has viewed the notification.	Default = false.	false

3.8.3 Database Schema

Step 1: Beginning from the unnormalized state

With regards to the case of data being gathered on the platform in an unnormalized manner, then in such a scenario, it would be possible to encounter the use of just one Claims table in which both the details of the claimant, that is the name and the email address, the claim information, and even the document information will be placed. In particular, each document has its own columns in which the file name, file URL, and type will be indicated (IBM, 2024).

Step 2: First Normal Form (1NF)

The requirement of 1NF requires every field of a column to contain a single value and not a group of values. Therefore, repeating document fields discussed above have been separated out into an independent Documents table, with each record referring back to a particular claim by means of ClaimId foreign key. This implies that any number of documents may be created against a claim without altering the schema of the Claims table (IBM, 2024).

Step 3: Second Normal Form (2NF)

For 2NF, a table should already be in 1NF and every non-key attribute should be dependent on whole of the primary key and not just part of it. This is especially relevant when we have a composite key (that is, a primary key consisting of multiple columns). However, since every table in the current database schema (Claims, Documents, Claim_Tracking and Notifications) uses a single-column primary key (ClaimId, DocumentId, TrackingId and NotificationId respectively), there is no issue of partial dependency (IBM, 2024).

Step 4: Third Normal Form (3NF)

For a table to be in 3NF, the following conditions must hold: firstly, the table has to be in 2NF, and secondly, there should be no transitive dependency – the situation when the table has a field that depends on another non-key field and not the primary key field. For example, had the Claim_Tracking table contained the administrator's name and email directly, instead of containing an AdminId, it would be a case of transitive dependency, since the administrator name and email would be dependent on the administrator and not on the tracking item (IBM, 2024).Therefore, the Claim_Tracking table does not have the administrator name and email, but it rather contains a foreign key (AdminId) referencing the Admins table where such information can be found. The same logic holds for Claims, which only contains the UserId (IBM, 2024).

As a result of the above changes, all six tables of the data dictionary (Users, Claims, Documents, Admins, Claim_Tracking, and Notifications) become Third Normal Form tables (IBM, 2024).

Table relationships summary

Users → Claims (1:M): Each user can have many claims and each claim is submitted by one user (IBM, 2024).

Claims → Documents (1:M) one claim has many documents, each document belongs to one claim (IBM, 2024).

Claims → Claim_Tracking (1:M) : One claim can have many tracking entries but each tracking entry belongs to only one claim (IBM, 2024).

Admins → Claim_Tracking (1:M) one admin can create many tracking entries, each tracking entry is associated with one admin (IBM, 2024).

Users → Notifications (1:M): One user can have many notifications, each notification is linked to one user (IBM, 2024).

3.8.4 Data Flow Diagram

Symbol	Name	Description
Rectangle	External Entity Symbol	Represents a person or system outside the platform that sends or receives data, such as the Claimant or the Administrator.
Circle	Process Symbol	Represents an activity that transforms incoming data into outgoing data. Every process must have at least one input and one output.
Open-ended rectangle (two horizontal lines)	Data Store Symbol	Represents a place where data is held for later use, such as a Firestore collection.
Arrow	Data Flow Symbol	Represents the movement of data between an external entity, a process and a data store. The arrowhead shows the direction of the flow.

Table 2: Data Flow Diagram Key

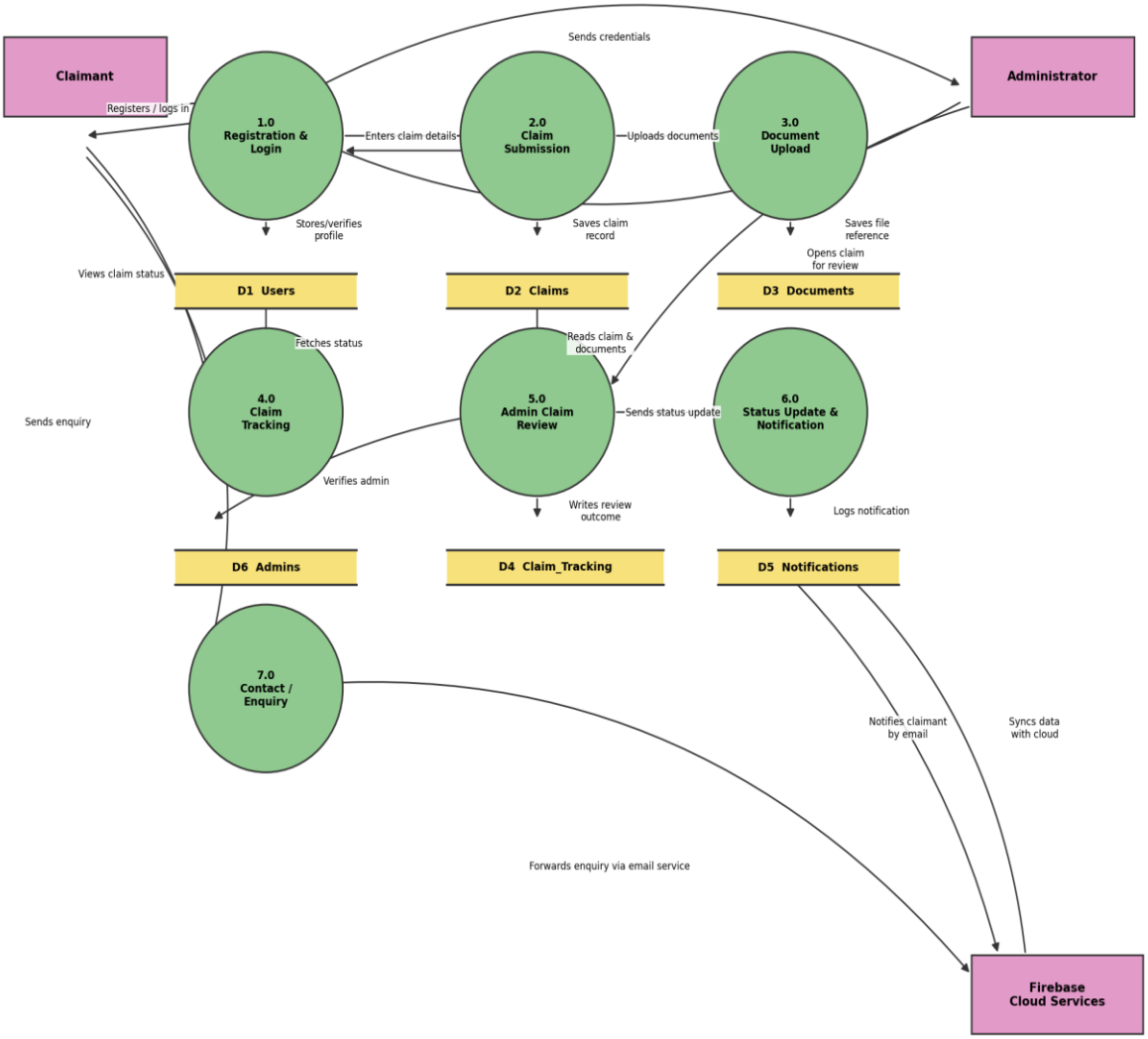


Figure 3.8.4: Data Flow Diagram for the Smart Funeral Claims Platform

As illustrated by the diagram, a Claimant registers and logs in using Process 1.0, which contains and validates the Claimant's profile in the Users data store. A Claimant initiates a claim using Process 2.0 and uploads relevant documents using Process 3.0 which stores the information in the Claims and Documents data stores respectively. A Claimant is able to track his/her claim using Process 4.0, Claim Tracking which reads information from the Claims data store (Lucidchart, 2026).

For the administration part, an administrator opens a claim for approval using Process 5.0, which reads claim and document information and authenticates the administrator in the

Admins data store. The decision made is stored in the Claim_Tracking data store using Process 6.0, which logs the notification in the Notifications data store and emails the claimant about the decision. There is also a Contact/Enquiry process (Process 7.0) where a Claimant raises a query which is then sent via email services configured. The Firebase Cloud Services sits behind all the data stores (Lucidchart, 2026).

Conclusion

This phase has been able to analyze the current way funeral claims have been managed, through either the manual method or the few web solutions that have been offered by various providers, and based on these findings, it has been able to come up with the requirements of the Smart Funeral Claims Platform. It has been seen from the observations, participatory inquiry, and casual interviews that the issues faced by the claimants and administrators mainly include the use of paperwork, lack of visibility of the status of the claim, and the manual processing of claims.

From the above findings, this stage formulated functional and non-functional requirements for the system and developed data models which include Entity Relationship Diagram, a complete data dictionary, a normalized database design, and a Data Flow Diagram. The above models provide an exact description of how the data for users, claims, documents, administrators, claims tracking, and notifications will be structured and how they will flow in the system.

REFERENCES

Asana. 2026. *Requirements gathering: 6 steps for project success*. [Online] Available at: https://asana.com/resources/requirements-gathering (Accessed: 7 September 2026).

GeeksforGeeks. 2026. *Functional and non-functional requirements*. [Online] Available at: https://www.geeksforgeeks.org/software-engineering/functional-vs-non-functional-requirements/ (Accessed: 6 September 2026).

GeeksforGeeks. 2025. *Non-functional requirements in software engineering*. [Online] Available at: https://www.geeksforgeeks.org/software-engineering/non-functional-requirements-in-software-engineering/ (Accessed: 2 September 2026).

IBM. 2022. *What is data modeling?* [Online] Available at: https://www.ibm.com/think/topics/data-modeling (Accessed: 1 September 2026).

IBM. 2026. *What is data modeling?* [Online] Available at: https://www.ibm.com/think/topics/data-modeling?no_head=1 (Accessed: 3 September 2026).

IBM. 2025. *What is a primary key?* [Online] Available at: https://www.ibm.com/think/topics/primary-key (Accessed: 28 August 2026).

IBM. 2024. *What is a database?* [Online] Available at: https://www.ibm.com/think/topics/database (Accessed: 30 August 2026).

IBM. 2024. *What is an entity relationship diagram?* [Online] Available at: https://www.ibm.com/think/topics/entity-relationship-diagram (Accessed: 4 September 2026).

IBM. 2025. *What is software development?* [Online] Available at: https://www.ibm.com/think/topics/software-development (Accessed: 25 August 2026).

IBM. 2025. *What is requirements management?* [Online] Available at: https://www.ibm.com/think/topics/what-is-requirements-management (Accessed: 27 August 2026).

Lucidchart. 2026. *A quick guide to different types of entity relationship diagrams*. [Online] Available at: https://www.lucidchart.com/blog/types-of-entity-relationship-diagrams (Accessed: 5 September 2026).

Oracle. 2026. *Maintaining data integrity in database applications*. [Online] Available at: https://docs.oracle.com/en/database/oracle/oracle-database/26/adfns/data-integrity.html (Accessed: 6 September 2026).

Oracle. 2025. *Data integrity*. [Online] Available at: https://docs.oracle.com/database/121/CNCPT/datainte.htm (Accessed: 29 August 2026).

OpenStax. 2025. *Systems Analysis and Design for Application Development*. [Online] Available at: https://openstax.org/books/foundations-information-systems/pages/4-1-systems-analysis-and-design-for-application-development (Accessed: 7 September 2026).